

QuickDraw CNN Classifier Report

Aminreza Safarpour

December 17, 2025

Abstract

This report documents the design, training, evaluation, and results of a Convolutional Neural Network (CNN) model built for classifying images from the QuickDraw dataset, which contains 100 different classes. The report covers the architecture of the model, training procedure, evaluation results, and a discussion of the model's strengths and weaknesses.

1 CNN Architecture Description

1.1 Model Overview

In this project, a Convolutional Neural Network (CNN) was designed to classify images from the QuickDraw dataset. The architecture consists of three convolutional layers, followed by max-pooling and fully connected layers. This model aims to extract features from the images and use those features for classification into 100 different categories.

1.2 Layer Details

The following layers were used in the architecture:

- **Convolutional Layer 1 (Conv1):**
 - Number of filters: 32
 - Kernel size: 3x3
 - Padding: 1
 - Activation function: ReLU
- **Convolutional Layer 2 (Conv2):**

- Number of filters: 64
- Kernel size: 3x3
- Padding: 1
- Activation function: ReLU
- **Convolutional Layer 3 (Conv3):**
 - Number of filters: 128
 - Kernel size: 3x3
 - Padding: 1
 - Activation function: ReLU
- **MaxPooling Layers:**
 - Pool size: 2x2 (After each convolutional layer)
- **Fully Connected Layer 1 (FC1):**
 - Number of neurons: 512
 - Activation function: ReLU
- **Fully Connected Layer 2 (FC2):**
 - Number of neurons: 100 (Output corresponding to 100 classes)

1.3 Explanation of Design Choices

- The **convolutional layers** are designed to progressively extract higher-level features from the images. ReLU activation functions are applied after each convolution to introduce non-linearity.
- **MaxPooling** helps reduce the spatial dimension of the feature maps, improving computational efficiency and helping prevent overfitting.
- The **fully connected layers** help combine features from all the convolutional layers into a final vector, which is used to predict the class of the image.

2 Training Procedure

2.1 Hyperparameters

The following hyperparameters were used for training the CNN:

- **Learning Rate:** 0.001
- **Optimizer:** Adam
- **Batch Size:** 32
- **Number of Epochs:** 10
- **Loss Function:** CrossEntropyLoss

2.2 Data Preprocessing

The images were preprocessed as follows:

- Converted to grayscale (1 channel) to reduce the computational cost.
- Resized to 28*28 pixels to match the input size required by the model.
- Normalized to the range $[-1, 1]$ using the formula $\frac{pixel_value - 0.5}{0.5}$.

2.3 Train/Test Split

The dataset was split into training and testing sets:

- **Training Data:** 80% of the total data
- **Testing Data:** 20% of the total data

3 Plots

3.1 Training vs. Test Loss

The following plot shows the training and test loss over the course of the training process:

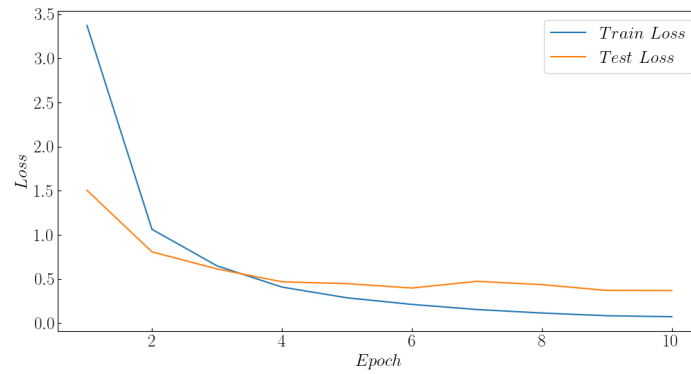


Figure 1: Training vs Test Loss

3.2 Training vs. Test Accuracy

This plot illustrates the training and test accuracy across epochs:

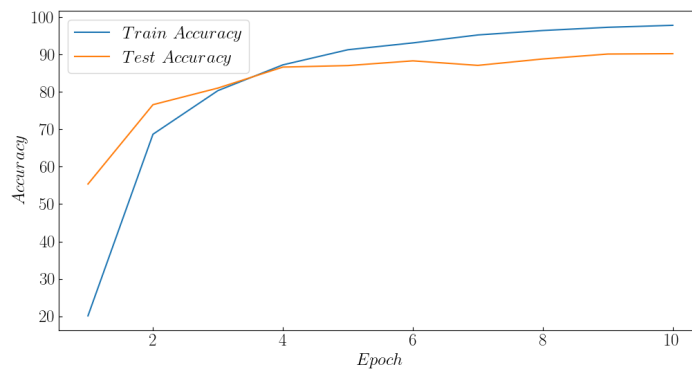


Figure 2: Training vs Test Accuracy

4 Results & Discussion

4.1 Final Test Accuracy

The final test accuracy achieved by the model was 90.20%, which indicates a strong performance on the validation set.

4.2 Observations about Underfitting/Overfitting

The model did not exhibit significant overfitting, as evidenced by the similar trends in training and test accuracy. There is also no evidence of underfitting since the model performed well on both the training and test data.

4.3 Strengths and Weaknesses

4.3.1 Strengths

- The model performed well with a simple architecture.
- Efficient computation due to the use of smaller images (28x28 pixels).

4.3.2 Weaknesses

- The model could face challenges with more complex or higher-resolution images.
- A deeper architecture could further improve performance.

4.4 Potential Improvements

- Implement data augmentation (e.g., rotation, flipping, zooming) to increase the variety of training data and improve generalization.
- Experiment with more advanced architectures such as ResNet or VGG.
- Hyperparameter tuning (e.g., different learning rates, batch sizes, etc.) could further improve the model's performance.

5 Conclusion

In conclusion, the CNN architecture designed for the QuickDraw dataset shows promising results with an 90.20% test accuracy. While the model

performed well on the given dataset, further improvements can be made by implementing more complex architectures and data augmentation techniques.